

TecNM Campus Matehuala

Ingeniería de Software

2.4

Diseño de Datos

TallerTec — Esquema de Base de Datos y Modelo Relacional

Proyecto: TallerTec

Sistema Integral de Gestión de Talleres Deportivos

Equipo de Desarrollo: Virgilio Power | Febrero 2025

1. Introducción

Este documento especifica el diseño completo de la base de datos de TallerTec, incluyendo el esquema relacional normalizado en Tercera Forma Normal (3FN), el DDL SQL completo listo para ejecutar en Supabase PostgreSQL, las políticas de seguridad RLS, los triggers, índices y las restricciones de integridad referencial.

El modelo de datos está diseñado para garantizar la consistencia, integridad y rendimiento necesarios para gestionar hasta 5,000 estudiantes, 20 talleres y 10,000+ registros de asistencia por semestre.

2. Diagrama Entidad-Relación



Figura 2.1 — Diagrama Entidad-Relación TallerTec (3FN)

2.1 Normalización — Verificación de 3FN

Tabla	Forma Normal	Justificación
usuarios	3FN	Sin dependencias transitivas. numero_control no determina otros atributos
periodos	3FN	Todos los atributos dependen únicamente de la PK (id)
talleres	3FN	responsable_id y periodo_id son FK, no hay dependencias parciales
inscripciones	3FN	horas_acumuladas calculado por trigger, no viola 3FN (valor derivado)
asistencias	3FN	horas_computadas depende directamente del id de asistencia
constancias	3FN	horas_totales es snapshot en tiempo de emisión (intencional)
notificaciones	3FN	Sin atributos derivados ni dependencias transitivas

3. DDL SQL — Esquema Completo

3.1 Tipos Enumerados (ENUM)

```
-- Tipos enumerados para dominios controlados
CREATE TYPE rol_enum AS ENUM (
  'ESTUDIANTE', 'RESPONSABLE_TALLER', 'ADMIN_OFICINA'
);

CREATE TYPE estado_inscripcion AS ENUM (
  'ACTIVA', 'BAJA', 'COMPLETADA'
);
```

```
CREATE TYPE metodo_registro AS ENUM ('QR', 'MANUAL');
```

```
CREATE TYPE estado_constancia AS ENUM (
    'PENDIENTE', 'APROBADA', 'RECHAZADA', 'GENERADA', 'ENTREGADA'
);
```

```
CREATE TYPE categoria_taller AS ENUM ('DEPORTIVO', 'CULTURAL');
```

```
CREATE TYPE tipo_notificacion AS ENUM (
    'INSCRIPCION', 'ASISTENCIA', 'CONSTANCIA_SOLICITUD',
    'CONSTANCIA_LISTA', 'CONSTANCIA_RECHAZADA', 'META_20H'
);
```

3.2 Tabla: usuarios

```
CREATE TABLE usuarios (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email             VARCHAR(255) NOT NULL UNIQUE
                    CHECK (email ~* '^[A-Za-z0-9._%+-]+@matehuala\.tecnm\.mx$'),
    nombre_completo   VARCHAR(200) NOT NULL CHECK (char_length(nombre_completo) >=
3),
    numero_control    VARCHAR(20) UNIQUE,
    rol               rol_enum NOT NULL DEFAULT 'ESTUDIANTE',
    telefono          VARCHAR(15),
    carrera           VARCHAR(100),
    semestre          INTEGER CHECK (semestre BETWEEN 1 AND 12),
    codigo_qr         TEXT UNIQUE,
    foto_url          TEXT,
    activo            BOOLEAN NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
COMMENT ON TABLE usuarios IS 'Usuarios del sistema: estudiantes, responsables y
admins';
COMMENT ON COLUMN usuarios.codigo_qr IS 'Token JWT único para identificación QR
del estudiante';
```

3.3 Tabla: periodos

```
CREATE TABLE periodos (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre            VARCHAR(100) NOT NULL UNIQUE,
    fecha_inicio      DATE NOT NULL,
    fecha_fin         DATE NOT NULL,
    inscripciones_abiertas  BOOLEAN NOT NULL DEFAULT FALSE,
    activo            BOOLEAN NOT NULL DEFAULT TRUE,
    created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
CONSTRAINT chk_fechas CHECK (fecha_fin > fecha_inicio)
);
COMMENT ON TABLE periodos IS 'Períodos semestrales: Enero-Junio, Agosto-Diciembre';
```

3.4 Tabla: talleres

```
CREATE TABLE talleres (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre            VARCHAR(150) NOT NULL,
  descripcion       TEXT,
  categoria         categoria_taller NOT NULL DEFAULT 'DEPORTIVO',
  horario_texto     VARCHAR(200) NOT NULL,
  ubicacion         VARCHAR(150) NOT NULL,
  cupo_maximo       INTEGER NOT NULL CHECK (cupo_maximo > 0),
  cupo_disponible   INTEGER NOT NULL CHECK (cupo_disponible >= 0),
  responsable_id    UUID NOT NULL REFERENCES usuarios(id),
  periodo_id        UUID NOT NULL REFERENCES periodos(id),
  activo           BOOLEAN NOT NULL DEFAULT TRUE,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_cupo CHECK (cupo_disponible <= cupo_maximo)
);
CREATE INDEX idx_talleres_periodo_activo ON talleres(periodo_id, activo);
COMMENT ON TABLE talleres IS 'Catálogo de talleres deportivos y culturales por período';
```

3.5 Tabla: inscripciones

```
CREATE TABLE inscripciones (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  estudiante_id     UUID NOT NULL REFERENCES usuarios(id),
  taller_id         UUID NOT NULL REFERENCES talleres(id),
  fecha_inscripcion TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  estado            estado_inscripcion NOT NULL DEFAULT 'ACTIVA',
  horas_acumuladas  NUMERIC(5,2) NOT NULL DEFAULT 0.00
                  CHECK (horas_acumuladas >= 0),
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  -- Evita que un estudiante se inscriba dos veces al mismo taller en el mismo
  periodo
  CONSTRAINT uq_inscripcion_periodo UNIQUE (
    estudiante_id,
    taller_id
  )
);
CREATE INDEX idx_inscripciones_estudiante ON inscripciones(estudiante_id);
CREATE INDEX idx_inscripciones_taller ON inscripciones(taller_id);
```

3.6 Tabla: asistencias

```
CREATE TABLE asistencias (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  inscripcion_id    UUID NOT NULL REFERENCES inscripciones(id),  
  fecha             DATE NOT NULL DEFAULT CURRENT_DATE,  
  hora_entrada      TIME NOT NULL,  
  hora_salida       TIME,  
  horas_computadas  NUMERIC(4,2) NOT NULL DEFAULT 0.00  
                  CHECK (horas_computadas >= 0 AND horas_computadas <= 24),  
  metodo_registro   metodo_registro NOT NULL DEFAULT 'QR',  
  registrado_por    UUID NOT NULL REFERENCES usuarios(id),  
  notas             TEXT,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  -- No se puede registrar dos veces al mismo estudiante en el mismo día y  
  taller  
  CONSTRAINT uq_asistencia_dia UNIQUE (inscripcion_id, fecha)  
);  
CREATE INDEX idx_asistencias_inscripcion ON asistencias(inscripcion_id);  
CREATE INDEX idx_asistencias_fecha ON asistencias(fecha);
```

3.7 Tabla: constancias

```
CREATE TABLE constancias (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  estudiante_id     UUID NOT NULL REFERENCES usuarios(id),  
  periodo_id        UUID NOT NULL REFERENCES periodos(id),  
  fecha_generacion  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  horas_totales     NUMERIC(5,2) NOT NULL CHECK (horas_totales >= 20),  
  archivo_url       TEXT,  
  folio             VARCHAR(50) UNIQUE,  
  generado_por      UUID REFERENCES usuarios(id),  
  estado            estado_constancia NOT NULL DEFAULT 'PENDIENTE',  
  observaciones     TEXT,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  -- Un estudiante solo puede tener una constancia por período  
  CONSTRAINT uq_constancia_periodo UNIQUE (estudiante_id, periodo_id)  
);  
CREATE INDEX idx_constancias_estudiante ON constancias(estudiante_id);  
CREATE INDEX idx_constancias_estado ON constancias(estado);
```

3.8 Tabla: notificaciones

```
CREATE TABLE notificaciones (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  usuario_id        UUID NOT NULL REFERENCES usuarios(id),  
  tipo              tipo_notificacion NOT NULL,  
  titulo            VARCHAR(200) NOT NULL,
```

```

    mensaje      TEXT NOT NULL,
    leida        BOOLEAN NOT NULL DEFAULT FALSE,
    metadata     JSONB,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_notificaciones_usuario ON notificaciones(usuario_id, leida);

```

4. Triggers y Funciones PostgreSQL

4.1 Calcular horas acumuladas automáticamente

```

CREATE OR REPLACE FUNCTION calcular_horas_acumuladas()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE inscripciones
    SET horas_acumuladas = (
        SELECT COALESCE(SUM(horas_computadas), 0)
        FROM asistencias
        WHERE inscripcion_id = NEW.inscripcion_id
    )
    WHERE id = NEW.inscripcion_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_calcular_horas
AFTER INSERT OR UPDATE ON asistencias
FOR EACH ROW
EXECUTE FUNCTION calcular_horas_acumuladas();

```

4.2 Verificar meta de 20 horas y notificar

```

CREATE OR REPLACE FUNCTION verificar_meta_20h()
RETURNS TRIGGER AS $$
BEGIN
    -- Si acaba de alcanzar las 20 horas
    IF NEW.horas_acumuladas >= 20 AND OLD.horas_acumuladas < 20 THEN
        -- Actualizar estado a COMPLETADA
        NEW.estado = 'COMPLETADA';
        -- Crear notificación para el estudiante
        INSERT INTO notificaciones (usuario_id, tipo, titulo, mensaje)
        VALUES (
            NEW.estudiante_id,
            'META_20H',
            '¡Felicidades! Completaste tus 20 horas',
            'Ya puedes solicitar tu constancia desde la app.'
        );
    );

```

```

    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_verificar_meta
    BEFORE UPDATE OF horas_acumuladas ON inscripciones
    FOR EACH ROW
    EXECUTE FUNCTION verificar_meta_20h();

```

4.3 Sincronizar usuario con auth.users de Supabase

```

CREATE OR REPLACE FUNCTION handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.usuarios (id, email, nombre_completo, rol)
    VALUES (
        NEW.id,
        NEW.email,
        COALESCE(NEW.raw_user_meta_data->>'nombre_completo', 'Sin nombre'),
        COALESCE(NEW.raw_user_meta_data->>'rol', 'ESTUDIANTE')::rol_enum
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

```

```

CREATE TRIGGER trg_new_user
    AFTER INSERT ON auth.users
    FOR EACH ROW
    EXECUTE FUNCTION handle_new_user();

```

4.4 Generar folio único de constancia

```

CREATE OR REPLACE FUNCTION generar_folio_constancia()
RETURNS TRIGGER AS $$
DECLARE
    v_anio    INTEGER := EXTRACT(YEAR FROM NOW());
    v_sem     INTEGER := CASE WHEN EXTRACT(MONTH FROM NOW()) <= 6 THEN 1 ELSE 2
END;
    v_seq     INTEGER;
BEGIN
    SELECT COUNT(*) + 1 INTO v_seq FROM constancias
        WHERE EXTRACT(YEAR FROM created_at) = v_anio;
    NEW.folio := FORMAT('TALL-%s-%s-%s',
        v_anio, v_sem, LPAD(v_seq::TEXT, 5, '0'));
    RETURN NEW;
END;

```

```
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_folio_constancia
BEFORE INSERT ON constancias
FOR EACH ROW
EXECUTE FUNCTION generar_folio_constancia();
```

4.5 Actualizar cupo disponible del taller

```
CREATE OR REPLACE FUNCTION update_cupo_disponible()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE talleres
        SET cupo_disponible = cupo_disponible - 1
        WHERE id = NEW.taller_id;
    ELSIF TG_OP = 'DELETE' OR
        (TG_OP = 'UPDATE' AND NEW.estado = 'BAJA' AND OLD.estado = 'ACTIVA') THEN
        UPDATE talleres
        SET cupo_disponible = cupo_disponible + 1
        WHERE id = COALESCE(NEW.taller_id, OLD.taller_id);
    END IF;
    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_cupo_inscripcion
AFTER INSERT OR UPDATE OF estado OR DELETE ON inscripciones
FOR EACH ROW EXECUTE FUNCTION update_cupo_disponible();
```

5. Políticas RLS Completas

5.1 Tabla usuarios

```
ALTER TABLE usuarios ENABLE ROW LEVEL SECURITY;
```

```
-- Cada usuario puede ver su propio perfil; admin ve todos
CREATE POLICY pol_usuarios_select ON usuarios FOR SELECT
USING (auth.uid() = id OR
    EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND rol='ADMIN_OFICINA'));
```

```
-- Solo puede actualizar su propio perfil
CREATE POLICY pol_usuarios_update ON usuarios FOR UPDATE
USING (auth.uid() = id);
```

5.2 Tabla talleres

```
ALTER TABLE talleres ENABLE ROW LEVEL SECURITY;
```

```
-- Todos los autenticados pueden ver talleres activos
CREATE POLICY pol_talleres_select ON talleres FOR SELECT
  USING (activo = TRUE OR
    EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND rol='ADMIN_OFICINA'));
```

```
-- Solo admin puede crear/modificar talleres
CREATE POLICY pol_talleres_write ON talleres FOR ALL
  USING(EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND
    rol='ADMIN_OFICINA'));
```

5.3 Tabla inscripciones

```
ALTER TABLE inscripciones ENABLE ROW LEVEL SECURITY;
```

```
-- Estudiante ve sus inscripciones; responsable ve las de su taller; admin ve todo
CREATE POLICY pol_inscripciones_select ON inscripciones FOR SELECT
  USING (
    estudiante_id = auth.uid() OR
    EXISTS(SELECT 1 FROM talleres t JOIN usuarios u ON u.id=auth.uid()
      WHERE t.id=inscripciones.taller_id AND t.responsable_id=auth.uid())
OR
    EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND rol='ADMIN_OFICINA')
  );
```

```
-- Estudiante puede inscribirse (INSERT) si hay cupo
CREATE POLICY pol_inscripciones_insert ON inscripciones FOR INSERT
  WITH CHECK (
    estudiante_id = auth.uid() AND
    EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND rol='ESTUDIANTE') AND
    EXISTS(SELECT 1 FROM talleres WHERE id=taller_id AND cupo_disponible > 0)
  );
```

5.4 Tabla constancias

```
ALTER TABLE constancias ENABLE ROW LEVEL SECURITY;
```

```
-- Estudiante ve sus constancias; admin ve todas
CREATE POLICY pol_constancias_select ON constancias FOR SELECT
  USING (estudiante_id = auth.uid() OR
    EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND rol='ADMIN_OFICINA'));
```

```
-- Estudiante puede solicitar (INSERT estado=PENDIENTE con horas >= 20)
CREATE POLICY pol_constancias_insert ON constancias FOR INSERT
  WITH CHECK (
```

```

    estudiante_id = auth.uid() AND
    (SELECT SUM(horas_acumuladas) FROM inscripciones WHERE
    estudiante_id=auth.uid()) >= 20
);

```

```

-- Solo admin puede aprobar/rechazar/generar (UPDATE)
CREATE POLICY pol_constancias_update ON constancias FOR UPDATE
    USING(EXISTS(SELECT 1 FROM usuarios WHERE id=auth.uid() AND
    rol='ADMIN_OFICINA'));

```

6. Datos Iniciales (Seeds)

6.1 Período inicial

```

INSERT INTO periodos (nombre, fecha_inicio, fecha_fin, inscripciones_abiertas,
activo)
VALUES ('Enero-Junio 2025', '2025-01-13', '2025-06-15', TRUE, TRUE);

```

6.2 Talleres ejemplo

```

-- (responsable_id se actualiza al crear las cuentas de responsables)
INSERT INTO talleres (nombre, categoria, horario_texto, ubicacion, cupo_maximo,
    cupo_disponible, responsable_id, periodo_id)
VALUES
    ('Fútbol Varonil','DEPORTIVO','Lunes y Miércoles 7:00-9:00 AM',
    'Cancha Principal', 25, 25, '<uuid_resp_1>', '<uuid_periodo>'),
    ('Voleibol','DEPORTIVO','Martes y Jueves 6:00-8:00 PM',
    'Cancha Techada', 20, 20, '<uuid_resp_2>', '<uuid_periodo>'),
    ('Básquetbol','DEPORTIVO','Viernes 3:00-5:00 PM',
    'Duela Deportiva', 15, 15, '<uuid_resp_3>', '<uuid_periodo>'),
    ('Ajedrez','CULTURAL','Martes 4:00-6:00 PM',
    'Sala de Usos Múltiples', 30, 30, '<uuid_resp_4>', '<uuid_periodo>'),
    ('Atletismo','DEPORTIVO','Lunes Miércoles Viernes 6:00-7:30 AM',
    'Pista Atlética', 20, 20, '<uuid_resp_5>', '<uuid_periodo>');

```

7. Resumen del Modelo de Datos

Tabla	Registros Est.	PK	FKs	Índices
usuarios	5,000–6,000	UUID	—	email, codigo_qr
periodos	2–4	UUID	—	—
talleres	15–25	UUID	usuarios, periodos	periodo_id+activo
inscripciones	5,000–7,500	UUID	usuarios, talleres	estudiante, taller
asistencias	50,000–80,000	UUID	inscripciones, usuarios	inscripcion, fecha

Tabla	Registros Est.	PK	FKs	Índices
constancias	1,000–2,000	UUID	usuarios, periodos, usuarios	estudiante, estado
notificaciones	15,000+	UUID	usuarios	usuario+leida

El modelo completo ocupa aproximadamente **45–80 MB** en el tier gratuito de Supabase (500 MB disponibles), con amplio margen para el volumen proyectado del campus Matehuala.